

# DEBACU

## HOJA DE RUTA API – INTEGRACIÓN CON PMS

Documento estratégico y técnico

Versión 1.0

Plataforma: **Debacu Evaluation / Revenue Intelligence**

---

### 1. Introducción

Debacu es una plataforma SaaS para el sector hotelero orientada a:

- análisis de riesgo operativo
- inteligencia de revenue
- monitorización de comportamiento de huéspedes
- análisis multi-propiedad
- detección de incidencias y patrones
- generación de alertas operativas

Para alimentar estos módulos, Debacu necesita acceder a datos operativos del hotel que se encuentran normalmente en el **Property Management System (PMS)**.

Este documento define la **estrategia de integración API de Debacu con PMS**, estableciendo:

- alcance funcional inicial
- requisitos habituales de los PMS
- arquitectura recomendada
- política de seguridad
- modelo de permisos
- hoja de ruta de implementación

El objetivo es **integrar Debacu con PMS de forma segura, escalable y aprobable por los proveedores**.

---

## 2. Principios estratégicos de integración

La estrategia inicial de Debacu se basa en los siguientes principios.

### 1. Integración progresiva

No intentar integrar todos los PMS desde el inicio.

Primero:

- 2 o 3 PMS estratégicos
  - validación con hoteles reales
  - mejora progresiva
- 

### 2. Integración solo lectura (read-only)

En la primera fase Debacu **solo leerá datos del PMS**.

Debacu **no modificará reservas ni operaciones del hotel**.

Esto reduce:

- fricción de aprobación
  - riesgos operativos
  - complejidad técnica
  - tiempos de certificación
- 

### 3. Mínimo privilegio

Debacu solicitará **solo los permisos estrictamente necesarios**.

Nunca se solicitarán permisos de escritura si no son necesarios.

---

### 4. Arquitectura desacoplada

Debacu no se diseñará alrededor de un PMS concreto.

Se construirá:

**una capa de integración abstracta**

con adaptadores para cada PMS.

Esto permite:

- escalar a nuevos PMS
- mantener independencia tecnológica

- evitar dependencias estructurales
- 

## 3. Caso de uso de Debacu para los PMS

Cuando Debacu solicite acceso a un PMS, el caso de uso debe presentarse de forma clara.

### Descripción funcional

Debacu es una plataforma SaaS de analítica hotelera que transforma datos operativos del PMS en indicadores de:

- riesgo operativo
- comportamiento de huéspedes
- revenue intelligence
- incidencias y patrones
- alertas operativas

Debacu consume datos del PMS **exclusivamente con fines analíticos**.

No modifica la operativa del hotel ni ejecuta acciones sobre el PMS en la fase inicial.

---

### Beneficios para el hotel

- mejor control del riesgo operativo
  - detección temprana de incidencias
  - análisis de revenue y comportamiento
  - cuadros de mando multi-propiedad
  - mejora en la toma de decisiones
- 

## 4. Datos que Debacu necesita del PMS

En la fase inicial Debacu necesita únicamente **datos de lectura**.

---

### 4.1 Reservas

Datos necesarios para análisis de ocupación, segmentación y revenue.

Campos típicos:

- reservation\_id
  - status
  - created\_at
  - check\_in\_date
  - check\_out\_date
  - nights
  - number\_of\_rooms
  - adults
  - children
  - rate\_plan
  - channel
  - segment
  - currency
  - total\_amount
  - cancellation\_status
- 

## 4.2 Huéspedes

Datos necesarios para análisis de comportamiento y riesgo.

Campos típicos:

- guest\_id
  - first\_name
  - last\_name
  - email
  - phone
  - country
  - nationality
  - birth\_date (si está disponible)
  - historical stays (si el PMS lo expone)
-

## 4.3 Estancias (stay data)

Para identificar estado operativo de reservas.

Campos típicos:

- stay\_id
  - check\_in\_status
  - check\_out\_status
  - arrival\_date
  - departure\_date
  - actual\_check\_in
  - actual\_check\_out
  - no\_show
  - in\_house
  - cancelled
- 

## 4.4 Habitaciones y room types

Para análisis operativo y revenue.

Campos típicos:

### **Room types**

- room\_type\_id
- name
- capacity
- description

### **Habitaciones**

- room\_id
  - room\_number
  - room\_type\_id
  - floor
  - operational\_status
-

## 4.5 Datos económicos (opcional)

Solo si es necesario para módulos de revenue.

Ejemplos:

- ADR
- revenue rooms
- folio summary
- charges summary

Recomendación:

evitar integrar **folios completos en la primera fase.**

---

## 5. Permisos API (Scopes mínimos)

Debacu solicitará solo permisos de lectura.

Permisos típicos:

reservations.read  
guests.read  
stays.read  
rooms.read  
room\_types.read  
properties.read

Opcional:

folios.read  
charges.read

---

Permisos que **no se solicitarán en fase inicial:**

reservations.write  
guests.write  
inventory.write  
rates.write  
charges.write  
payments.write  
housekeeping.write

Esto facilita la aprobación por parte del PMS.

---

## 6. Arquitectura de autenticación

Cada PMS utiliza un modelo diferente.

Los más comunes son:

- OAuth2

- API Key
  - Client credentials
  - Tokens emitidos por la propiedad
  - Tokens emitidos por el proveedor
- 

## Arquitectura interna recomendada para Debacu

Debacu debe soportar múltiples modelos de autenticación mediante una capa común.

Cada integración PMS se almacenará con:

```
org_id
property_id
pms_provider
environment
credentials
token_expiration
sync_status
```

---

## Seguridad de credenciales

Las credenciales deben:

- almacenarse cifradas
  - gestionarse desde backend
  - nunca exponerse al frontend
  - rotarse cuando el PMS lo permita
- 

## Regla fundamental

Las llamadas al PMS **nunca se harán desde el navegador**.

Siempre desde:

- backend
  - Edge Functions
  - servicios seguros
- 

# 7. Política de seguridad Debacu

La política de seguridad debe cubrir:

## Protección de credenciales

- cifrado de tokens
  - acceso restringido
  - rotación de secretos
- 

## Seguridad de acceso

- autenticación de usuarios
  - autorización por organización
  - autorización por propiedad
  - separación multi-tenant estricta
- 

## Seguridad de datos

- cifrado HTTPS
  - cifrado en reposo
  - minimización de datos
  - acceso restringido
- 

## Auditoría

Debacu debe registrar:

- conexiones PMS
- sincronizaciones
- errores
- accesos
- webhooks recibidos

Cada evento debe registrar:

org\_id  
property\_id  
pms\_provider  
timestamp  
operation  
status

---



## Cumplimiento y privacidad

Debacu aplicará:

- RGPD
  - minimización de datos
  - control de retención
  - capacidad de desconexión PMS
  - borrado de credenciales
- 

## 8. Webhooks y callbacks

Muchos PMS requieren declarar URLs de callback.

Debacu debe preparar:

### **OAuth redirect URI**

/integrations/pms/oauth/callback

---

### **Webhook endpoint**

/integrations/pms/webhook

---

### **Validaciones necesarias**

- verificación de firma
  - control de origen
  - registro de eventos
  - reintentos
- 

### **Entornos**

Debacu debe separar:

sandbox  
production

---

## 9. Soporte técnico

Debacu debe definir claramente:

### **Contacto técnico**

email técnico

### **Contacto soporte**

email soporte

### **Responsable técnico**

arquitectura

### **Responsable funcional**

producto

---

Tiempo de respuesta recomendado:

- incidentes críticos: 24h
  - incidencias normales: 48h
- 

## **10. Documentación para PMS Marketplace**

Muchos PMS requieren documentación pública.

Debacu debe preparar:

### **Ficha de integración**

Contenido:

- descripción de Debacu
  - beneficios
  - datos consumidos
  - tipo de integración
  - PMS compatibles
  - soporte
- 

### **Guía de conexión**

Debe explicar:

1. cómo conectar Debacu
2. permisos necesarios

3. configuración PMS
  4. sincronización inicial
  5. resolución de problemas
- 

## 11. Certificación de integraciones

Algunos PMS exigirán:

- revisión técnica
- revisión de seguridad
- pruebas funcionales
- validación OAuth
- revisión documentación
- validación UX
- pruebas en sandbox

Los PMS más grandes pueden exigir además:

- programa de partners
  - NDA
  - publicación en marketplace
  - certificación formal
- 

## 12. Arquitectura de integración Debacu

Debacu implementará una **capa de integración PMS modular**.

Arquitectura lógica:

```
PMSConnector (base)
├── ApaleoConnector
├── CloudbedsConnector
├── MewsConnector
├── OperaConnector
└── Otros
```

---

### Flujo de datos

```
PMS API
  ↓
Connector
  ↓
```

Normalización  
↓  
Modelo Debacu  
↓  
Base de datos  
↓  
Analytics / Risk / Revenue

---

## **Sincronización**

Debacu soportará:

### **Sincronización inicial**

descarga completa de datos.

### **Sincronización incremental**

solo cambios nuevos.

---

# **13. Hoja de ruta de implementación**

## **Fase 1**

Definir modelo interno Debacu para:

- reservas
  - huéspedes
  - estancias
  - habitaciones
  - room types
  - propiedades
- 

## **Fase 2**

Construir capa de conectores PMS.

---

## **Fase 3**

Implementar primer conector read-only.

---

## **Fase 4**

Realizar pruebas con hotel piloto.

---

## **Fase 5**

Documentar integración.

---

## **Fase 6**

Solicitar aprobación PMS.

---

# **14. Estrategia de despliegue**

No integrar todos los PMS a la vez.

Orden recomendado:

1 PMS inicial

2 PMS adicionales

pilotos reales

expansión progresiva

---

# **15. Conclusión**

La estrategia de Debacu debe ser:

- comenzar con integraciones simples
- solicitar permisos mínimos
- diseñar arquitectura desacoplada
- validar valor real con hoteles
- escalar después

Intentar integrar todo desde el inicio sería un error estratégico.

---

# 16. Uso de este documento con Claude Code

Este documento puede utilizarse como **base de conocimiento para Claude Code**.

Claude debe usarlo como referencia para:

- diseñar conectores PMS
- definir modelos de datos
- proponer arquitectura de integración
- generar código de conectores
- validar permisos API
- revisar seguridad de integraciones

Claude debe respetar los principios:

- integración read-only inicial
- mínimo privilegio
- seguridad por defecto
- arquitectura desacoplada
- multi-propiedad